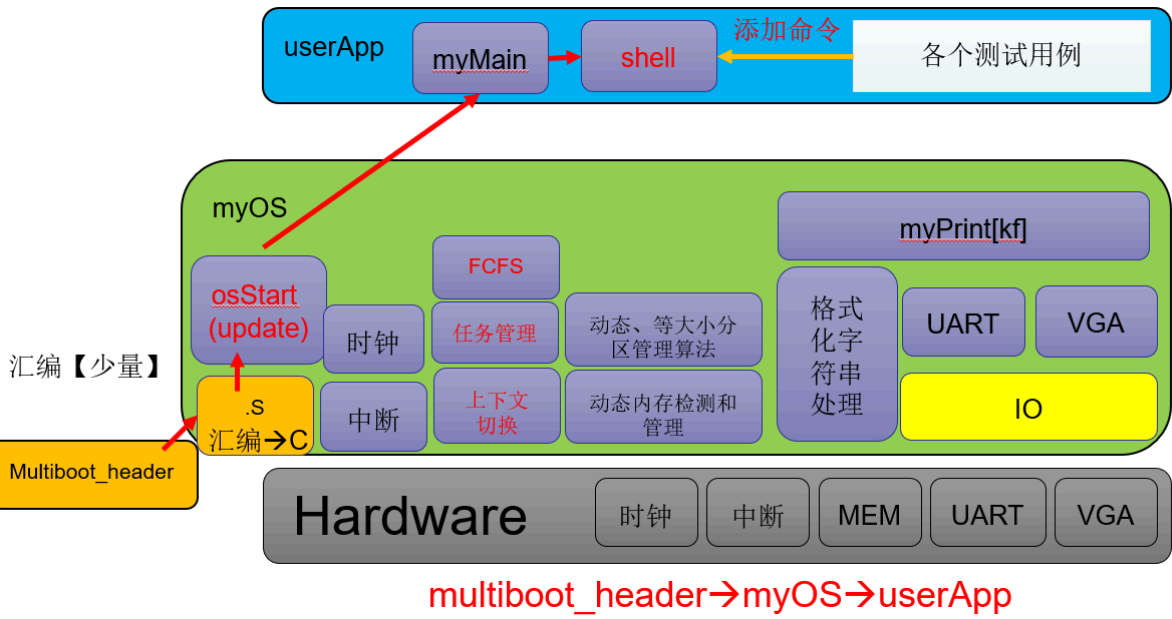


Lab5实验报告

李毅PB22051301

第一部分 系统布局



系统布局与lab5保持一致，调度算法使用FCFS，SJF和RR。

本次改动或新增模块：

任务参数模块：

- taskPara，可设置任务的参数，包含到达时间，优先级，执行时间

任务队列数据结构：

- taskQueueFIFO：TCB的先进先出队列
- taskQueuePrio：TCB的优先队列（出队元素是最小元素/最大元素，需指定比较函数）

任务到达模块：

- taskArrv，基于优先队列和tick实现

调度算法模块：

- schedulerFCFS，FCFS调度，基于先进先出队列实现
- schedulerSJF，非抢占SJF调度，基于优先队列实现
- schedulerPrio，非抢占优先级调度，允许每个优先级有多个任务，基于优先队列实现
- schedulerRR，时间片轮转调度，基于先进先出队列和tick实现

调度器抽象接口：

- scheduler，基于以上四种调度算法模块实现，向任务管理器提供统一接口

任务管理器模块：

- task，基于任务到达模块、任务参数模块、调度器接口实现

第二部分 代码解释

1. 任务参数模块 (taskPara.h / taskPara.c)

结构体定义：

```
typedef struct taskPara {  
    unsigned long arrivalTime;    /* 任务到达时间 */  
    unsigned long burstTime;      /* 任务执行时间(用于SJF) */  
    unsigned long remainingTime;  /* 剩余执行时间 */  
    unsigned long priority;       /* 任务优先级 */  
    unsigned long startTime;      /* 任务开始时间 */  
    unsigned long endTime;        /* 任务结束时间 */  
    unsigned long waitTime;       /* 任务等待时间 */  
} taskPara;
```

函数实现：

```
#include "../include/taskPara.h"  
  
/* 初始化任务参数 */  
void taskParaInit(taskPara *para, unsigned long arrivalTime,  
                  unsigned long burstTime, unsigned long priority) {  
    para->arrivalTime = arrivalTime;  
    para->burstTime = burstTime;  
    para->remainingTime = burstTime;  
    para->priority = priority;  
    para->startTime = 0;  
    para->endTime = 0;  
    para->waitTime = 0;  
}  
  
/* 设置任务参数 */  
void setTaskArrivalTime(taskPara *para, unsigned long arrivalTime) {  
    para->arrivalTime = arrivalTime;  
}  
  
void setTaskBurstTime(taskPara *para, unsigned long burstTime) {  
    para->burstTime = burstTime;  
    para->remainingTime = burstTime;  
}  
  
void setTaskPriority(taskPara *para, unsigned long priority) {  
    para->priority = priority;  
}  
  
/* 获取任务参数 */  
unsigned long getTaskArrivalTime(taskPara *para) {  
    return para->arrivalTime;  
}  
  
unsigned long getTaskBurstTime(taskPara *para) {  
    return para->burstTime;  
}
```

```

unsigned long getTaskRemainingTime(taskPara *para) {
    return para->remainingTime;
}

unsigned long getTaskPriority(taskPara *para) {
    return para->priority;
}

```

2.任务队列数据结构模块 (taskQueue.h/taskQueue.c)

使用两种队列，先进先出队列（FIFO和RR）和优先队列（SJF）。

结构体定义：

```

/* 先进先出队列结构 */
typedef struct taskQueueFIFO {
    struct myTCB *head;
    struct myTCB *tail;
    int size;
} taskQueueFIFO;

/* 优先队列结构 */
typedef struct taskQueuePrio {
    struct myTCB **heap;
    int size;
    int capacity;
    int (*compare)(struct myTCB *a, struct myTCB *b); /* 比较函数指针 */
} taskQueuePrio;

```

函数实现：

```

/* FIFO队列操作实现 */
void taskQueueFIFOInit(taskQueueFIFO *queue) {
    queue->head = NULL;
    queue->tail = NULL;
    queue->size = 0;
}

int taskQueueFIFOIsEmpty(taskQueueFIFO *queue) {
    return queue->head == NULL;
}

void taskQueueFIFOEnqueue(taskQueueFIFO *queue, struct myTCB *task) {
    task->nextTCB = NULL;

    if (taskQueueFIFOIsEmpty(queue)) {
        queue->head = task;
        queue->tail = task;
    } else {
        queue->tail->nextTCB = task;
        queue->tail = task;
    }
    queue->size++;
}

```

```

struct myTCB *taskQueueFIFODequeue(taskQueueFIFO *queue) {
    if (taskQueueFIFOIsEmpty(queue)) {
        return NULL;
    }

    struct myTCB *task = queue->head;
    queue->head = task->nextTCB;

    if (queue->head == NULL) {
        queue->tail = NULL;
    }

    task->nextTCB = NULL;
    queue->size--;
    return task;
}

struct myTCB *taskQueueFIFO Peek(taskQueueFIFO *queue) {
    return queue->head;
}

int taskQueueFIFOSize(taskQueueFIFO *queue) {
    return queue->size;
}

/* 优先队列操作实现 */
static struct myTCB *prioHeapArray[32]; /* 静态数组作为堆存储, 使用固定大小 */

void taskQueuePrioInit(taskQueuePrio *queue, int capacity, int (*compare)(struct
myTCB *a, struct myTCB *b)) {
    queue->heap = prioHeapArray;
    queue->size = 0;
    queue->capacity = capacity > 32 ? 32 : capacity;
    queue->compare = compare;
}

int taskQueuePrioIsEmpty(taskQueuePrio *queue) {
    return queue->size == 0;
}

/* 堆的上浮操作 */
static void heapifyUp(taskQueuePrio *queue, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (queue->compare(queue->heap[index], queue->heap[parent]) >= 0) {
            break;
        }

        /* 交换元素 */
        struct myTCB *temp = queue->heap[index];
        queue->heap[index] = queue->heap[parent];
        queue->heap[parent] = temp;

        index = parent;
    }
}

```

```

}

/* 堆的下沉操作 */
static void heapifyDown(taskQueuePrio *queue, int index) {
    while (1) {
        int smallest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        if (left < queue->size &&
            queue->compare(queue->heap[left], queue->heap[smallest]) < 0) {
            smallest = left;
        }

        if (right < queue->size &&
            queue->compare(queue->heap[right], queue->heap[smallest]) < 0) {
            smallest = right;
        }

        if (smallest == index) {
            break;
        }

        /* 交换元素 */
        struct myTCB *temp = queue->heap[index];
        queue->heap[index] = queue->heap[smallest];
        queue->heap[smallest] = temp;

        index = smallest;
    }
}

void taskQueuePrioEnqueue(taskQueuePrio *queue, struct myTCB *task) {
    if (queue->size >= queue->capacity) {
        myPrintk(0x2, "Priority queue is full!\n");
        return;
    }

    queue->heap[queue->size] = task;
    heapifyUp(queue, queue->size);
    queue->size++;
}

struct myTCB *taskQueuePrioDequeue(taskQueuePrio *queue) {
    if (taskQueuePrioIsEmpty(queue)) {
        return NULL;
    }

    struct myTCB *task = queue->heap[0];
    queue->heap[0] = queue->heap[queue->size - 1];
    queue->size--;

    if (queue->size > 0) {
        heapifyDown(queue, 0);
    }
}

```

```

        return task;
    }

    struct myTCB *taskQueuePrioPeek(taskQueuePrio *queue) {
        if (taskQueuePrioIsEmpty(queue)) {
            return NULL;
        }
        return queue->heap[0];
    }

    int taskQueuePrioSize(taskQueuePrio *queue) {
        return queue->size;
    }

```

辅助的比较函数实现如下:

```

/* 比较函数实现 */
int compareBurstTime(struct myTCB *a, struct myTCB *b) {
    unsigned long burstA = a->taskParam.burstTime;
    unsigned long burstB = b->taskParam.burstTime;

    if (burstA < burstB) return -1;
    if (burstA > burstB) return 1;
    return 0;
}

int comparePriority(struct myTCB *a, struct myTCB *b) {
    unsigned long prioA = a->taskParam.priority;
    unsigned long prioB = b->taskParam.priority;

    /* 优先级数值越小, 优先级越高 */
    if (prioA < prioB) return -1;
    if (prioA > prioB) return 1;
    return 0;
}

int compareArrivalTime(struct myTCB *a, struct myTCB *b) {
    unsigned long arrivalA = a->taskParam.arrivalTime;
    unsigned long arrivalB = b->taskParam.arrivalTime;

    if (arrivalA < arrivalB) return -1;
    if (arrivalA > arrivalB) return 1;
    return 0;
}

```

3. 任务到达模块 (taskArrv.h/taskArrv.c)

结构体定义:

```

/* 任务到达管理器 */
typedef struct taskArrvManager {
    taskQueuePrio waitingQueue; /* 等待到达的任务队列, 按到达时间排序 */
    taskQueueFIFO arrivedQueue; /* 已到达的任务队列 */
} taskArrvManager;

```

函数实现:

```
unsigned long systemTick = 0; /* 系统时钟计数 */

void taskArrvInit(taskArrvManager *manager) {
    taskQueuePrioInit(&(manager->waitingQueue), TASK_NUM, compareArrivalTime);
    taskQueueFIFOInit(&(manager->arrivedQueue));
}

void taskArrvAddTask(taskArrvManager *manager, struct myTCB *task) {
    taskQueuePrioEnqueue(&(manager->waitingQueue), task);
}

void taskArrvUpdate(taskArrvManager *manager) {
    while (!taskQueuePrioIsEmpty(&(manager->waitingQueue))) {
        struct myTCB *nextTask = taskQueuePrioPeek(&(manager->waitingQueue));
        if (nextTask->taskParam.arrivalTime <= systemTick) {
            taskQueuePrioDequeue(&(manager->waitingQueue));
            taskQueueFIFOEnqueue(&(manager->arrivedQueue), nextTask);
        } else {
            break;
        }
    }
}

struct myTCB *taskArrvGetNext(taskArrvManager *manager) {
    return taskQueueFIFODequeue(&(manager->arrivedQueue));
}

int taskArrvHasArrived(taskArrvManager *manager) {
    return !taskQueueFIFOIsEmpty(&(manager->arrivedQueue));
}
```

4. 调度算法模块 (schedulers.h/schedulers.c)

结构体定义:

```
/* FCFS调度器 */
typedef struct schedulerFCFS {
    taskQueueFIFO readyQueue;
} schedulerFCFS;

/* SJF调度器 */
typedef struct schedulerSJF {
    taskQueuePrio readyQueue;
} schedulerSJF;

/* 优先级调度器 */
typedef struct schedulerPrio {
    taskQueuePrio readyQueue;
} schedulerPrio;

/* RR调度器 */
typedef struct schedulerRR {
```

```

    taskQueueFIFO readyQueue;
    unsigned long timeQuantum;    /* 时间片大小 */
    unsigned long currentQuantum; /* 当前任务已用时间片 */
} schedulerRR;

```

函数实现:

```

/* FCFS调度器实现 */
void schedulerFCFSInit(schedulerFCFS *sched) {
    taskQueueFIFOInit(&(sched->readyQueue));
}

void schedulerFCFSAddTask(schedulerFCFS *sched, myTCB *task) {
    taskQueueFIFOEnqueue(&(sched->readyQueue), task);
}

myTCB *schedulerFCFSGetNext(schedulerFCFS *sched) {
    return taskQueueFIFODequeue(&(sched->readyQueue));
}

int schedulerFCFSIsEmpty(schedulerFCFS *sched) {
    return taskQueueFIFOIsEmpty(&(sched->readyQueue));
}

/* SJF调度器实现 */
void schedulerSJFInit(schedulerSJF *sched) {
    taskQueuePrioInit(&(sched->readyQueue), TASK_NUM, compareBurstTime);
}

void schedulerSJFAddTask(schedulerSJF *sched, myTCB *task) {
    taskQueuePrioEnqueue(&(sched->readyQueue), task);
}

myTCB *schedulerSJFGetNext(schedulerSJF *sched) {
    return taskQueuePrioDequeue(&(sched->readyQueue));
}

int schedulerSJFIsEmpty(schedulerSJF *sched) {
    return taskQueuePrioIsEmpty(&(sched->readyQueue));
}

/* 优先级调度器实现 */
void schedulerPrioInit(schedulerPrio *sched) {
    taskQueuePrioInit(&(sched->readyQueue), TASK_NUM, comparePriority);
}

void schedulerPrioAddTask(schedulerPrio *sched, myTCB *task) {
    taskQueuePrioEnqueue(&(sched->readyQueue), task);
}

myTCB *schedulerPrioGetNext(schedulerPrio *sched) {
    return taskQueuePrioDequeue(&(sched->readyQueue));
}

int schedulerPrioIsEmpty(schedulerPrio *sched) {

```



```

        return taskQueuePrioIsEmpty(&(sched->readyQueue));
    }

    /* RR调度器实现 */
    void schedulerRRInit(schedulerRR *sched, unsigned long quantum) {
        taskQueueFIFOInit(&(sched->readyQueue));
        sched->timeQuantum = quantum;
        sched->currentQuantum = 0;
    }

    void schedulerRRAddTask(schedulerRR *sched, myTCB *task) {
        taskQueueFIFOEnqueue(&(sched->readyQueue), task);
    }

    myTCB *schedulerRRGetNext(schedulerRR *sched) {
        sched->currentQuantum = 0; /* 重置时间片计数 */
        return taskQueueFIFODequeue(&(sched->readyQueue));
    }

    int schedulerRRIsEmpty(schedulerRR *sched) {
        return taskQueueFIFOIsEmpty(&(sched->readyQueue));
    }

    void schedulerRRTick(schedulerRR *sched) {
        sched->currentQuantum++;
    }

    int schedulerRRNeedPreempt(schedulerRR *sched) {
        return (sched->currentQuantum >= sched->timeQuantum) &&
            (!taskQueueFIFOIsEmpty(&(sched->readyQueue)));
    }
}

```

5. 调度器抽象接口 (scheduler.h/scheduler.c)

结构体定义：

```

/* 统一调度器接口 */
typedef struct scheduler {
    schedType type;
    union {
        schedulerFCFS fcfs;
        schedulerSJF sjf;
        schedulerPrio prio;
        schedulerRR rr;
    } impl;

    taskArrvManager arrvManager; /* 任务到达管理器 */
    struct myTCB *idleTask;      /* idle任务 */
} scheduler;

```

函数实现：

```

void schedulerInit(scheduler *sched, schedType type, unsigned long timeQuantum) {
    sched->type = type;
}

```

```

switch (type) {
    case SCHED_FCFS:
        schedulerFCFSInit(&(sched->impl.fcfs));
        break;
    case SCHED_SJF:
        schedulerSJFInit(&(sched->impl.sjf));
        break;
    case SCHED_PRIO:
        schedulerPrioInit(&(sched->impl.prio));
        break;
    case SCHED_RR:
        schedulerRRInit(&(sched->impl.rr), timeQuantum);
        break;
}

taskArrvInit(&(sched->arrvManager));
sched->idleTask = NULL;
}

void schedulerAddTask(scheduler *sched, myTCB *task) {
    /* 如果任务有到达时间, 先加入到达管理器 */
    if (task->taskParam.arrivalTime > systemTick) {
        taskArrvAddTask(&(sched->arrvManager), task);
        return;
    }

    /* 否则直接加入就绪队列 */
    switch (sched->type) {
        case SCHED_FCFS:
            schedulerFCFSAddTask(&(sched->impl.fcfs), task);
            break;
        case SCHED_SJF:
            schedulerSJFAddTask(&(sched->impl.sjf), task);
            break;
        case SCHED_PRIO:
            schedulerPrioAddTask(&(sched->impl.prio), task);
            break;
        case SCHED_RR:
            schedulerRRAddTask(&(sched->impl.rr), task);
            break;
    }
}

myTCB *schedulerGetNextTask(scheduler *sched) {
    myTCB *nextTask = NULL;

    /* 首先检查是否有新到达的任务 */
    taskArrvUpdate(&(sched->arrvManager));
    while (taskArrvHasArrived(&(sched->arrvManager))) {
        myTCB *arrivedTask = taskArrvGetNext(&(sched->arrvManager));
        schedulerAddTask(sched, arrivedTask);
    }

    /* 从相应的调度器获取下一个任务 */
    switch (sched->type) {

```

```

        case SCHED_FCFS:
            nextTask = schedulerFCFSGetNext(&(sched->impl.fcfs));
            break;
        case SCHED_SJF:
            nextTask = schedulerSJFGetNext(&(sched->impl.sjf));
            break;
        case SCHED_PRIO:
            nextTask = schedulerPrioGetNext(&(sched->impl.prio));
            break;
        case SCHED_RR:
            nextTask = schedulerRRGetNext(&(sched->impl.rr));
            break;
    }

    /* 如果没有就绪任务, 返回idle任务 */
    if (nextTask == NULL) {
        nextTask = sched->idleTask;
    }

    return nextTask;
}

void schedulerTick(scheduler *sched) {
    systemTick++;

    if (sched->type == SCHED_RR) {
        schedulerRRTick(&(sched->impl.rr));
    }
}

int schedulerNeedPreempt(scheduler *sched) {
    if (sched->type == SCHED_RR) {
        return schedulerRRNeedPreempt(&(sched->impl.rr));
    }
    return 0; /* 其他调度算法不支持抢占 */
}

void schedulerSetIdleTask(scheduler *sched, myTCB *idleTask) {
    sched->idleTask = idleTask;
}

void schedulerTaskComplete(scheduler *sched, myTCB *task) {
    /* 任务完成时的清理工作 */
    task->taskParam.endTime = systemTick;
    /* 这里可以添加更多的任务完成处理逻辑 */
}

```

6.修改现有的 task.c 文件, 集成新的调度器

```

/* 新增: 全局调度器实例 */
scheduler globalScheduler;
schedType currentSchedType = SCHED_FCFS; /* 默认使用FCFS调度 */

//新增: 带参数的任务创建函数
int createTskWithParam(void (*tskBody)(void), unsigned long arrivalTime,

```

```

        unsigned long burstTime, unsigned long priority) {
    if(firstFreeTsk == NULL) //没有可用的TCB
    {
        myPrintk(0x2, "No more TCB available!\n");
        return -1;
    }

    myTCB *newTsk = firstFreeTsk;
    firstFreeTsk = firstFreeTsk->nextTCB;

    newTsk->task_entrance = tskBody; //设置任务入口
    newTsk->TSK_State = TSK_NONE;
    stack_init(&(newTsk->stkTop), tskBody);

    /* 初始化任务参数 */
    taskParaInit(&(newTsk->taskParam), arrivalTime, burstTime, priority);

    tskStart(newTsk);

    return newTsk->TSK_ID;
}

//新增：统一调度函数
void scheduleUnified(void) {
    myTCB *nextTsk = schedulerGetNextTask(&globalScheduler);
    if (nextTsk != NULL && nextTsk != currentTsk) {
        context_switch(currentTsk, nextTsk);
    }
}

//调度算法（无需填写）
void schedule(void) {
    if (currentSchedType == SCHED_FCFS) {
        scheduleFCFS(); /* 兼容原有FCFS调度 */
    } else {
        scheduleUnified(); /* 使用新调度器 */
    }
}

/* 新增：调度算法切换和测试接口 */

// 切换调度算法
void setSchedulingAlgorithm(schedType type, unsigned long timeQuantum) {
    currentSchedType = type;
    schedulerInit(&globalScheduler, type, timeQuantum);
    schedulerSetIdleTask(&globalScheduler, idleTsk);
    myPrintk(0x2, "Scheduling algorithm changed to: ");
    switch(type) {
        case SCHED_FCFS:
            myPrintk(0x2, "FCFS\n");
            break;
        case SCHED_SJF:
            myPrintk(0x2, "SJF\n");
            break;
        case SCHED_PRIO:
            myPrintk(0x2, "Priority\n");

```

```

        break;
    case SCHED_RR:
        myPrintk(0x2, "Round Robin (quantum=%d)\n", timeQuantum);
        break;
    }
}

// 获取当前调度算法类型
schedType getCurrentSchedulingAlgorithm(void) {
    return currentSchedType;
}

// 创建测试任务的便捷接口
int createTestTask(void (*taskBody)(void), const char* name,
                  unsigned long arrivalTime, unsigned long burstTime,
                  unsigned long priority) {
    myPrintk(0x2, "Creating task: %s (arrival=%d, burst=%d, priority=%d)\n",
            name, arrivalTime, burstTime, priority);
    return createTskWithParam(taskBody, arrivalTime, burstTime, priority);
}

```

7.测试用例文件(scheduleTest.c)

```

#include "../include/task.h"
#include "../include/myPrintk.h"

/* 测试任务函数 */
void testTask1(void) {
    int i;
    for (i = 0; i < 3; i++) {
        myPrintk(0x3, "Task1 executing - iteration %d\n", i);
        // 简单的忙等待模拟任务执行
        volatile int j;
        for (j = 0; j < 100000; j++);
    }
    myPrintk(0x3, "Task1 completed\n");
}

void testTask2(void) {
    int i;
    for (i = 0; i < 2; i++) {
        myPrintk(0x4, "Task2 executing - iteration %d\n", i);
        volatile int j;
        for (j = 0; j < 150000; j++);
    }
    myPrintk(0x4, "Task2 completed\n");
}

void testTask3(void) {
    int i;
    for (i = 0; i < 4; i++) {
        myPrintk(0x5, "Task3 executing - iteration %d\n", i);
        volatile int j;
        for (j = 0; j < 80000; j++);
    }
}

```

```

    }
    myPrintk(0x5, "Task3 completed\n");
}

void testTask4(void) {
    int i;
    for (i = 0; i < 1; i++) {
        myPrintk(0x6, "Task4 executing - iteration %d\n", i);
        volatile int j;
        for (j = 0; j < 200000; j++);
    }
    myPrintk(0x6, "Task4 completed\n");
}

/* 测试FCFS调度算法 */
void testFCFS(void) {
    myPrintk(0x2, "\n=== Testing FCFS Scheduling ===\n");
    setSchedulingAlgorithm(SCHED_FCFS, 0);

    createTestTask(testTask1, "Task1", 0, 10, 1);
    createTestTask(testTask2, "Task2", 1, 8, 2);
    createTestTask(testTask3, "Task3", 2, 6, 3);
    createTestTask(testTask4, "Task4", 3, 12, 1);
}

/* 测试SJF调度算法 */
void testSJF(void) {
    myPrintk(0x2, "\n=== Testing SJF Scheduling ===\n");
    setSchedulingAlgorithm(SCHED_SJF, 0);

    // 创建不同执行时间的任务来测试SJF
    createTestTask(testTask1, "Task1", 0, 10, 1); // 长任务
    createTestTask(testTask2, "Task2", 1, 4, 2); // 短任务
    createTestTask(testTask3, "Task3", 2, 6, 3); // 中等任务
    createTestTask(testTask4, "Task4", 3, 2, 1); // 最短任务
}

/* 测试优先级调度算法 */
void testPriority(void) {
    myPrintk(0x2, "\n=== Testing Priority Scheduling ===\n");
    setSchedulingAlgorithm(SCHED_PRIO, 0);

    // 创建不同优先级的任务（数值越小优先级越高）
    createTestTask(testTask1, "Task1", 0, 10, 3); // 低优先级
    createTestTask(testTask2, "Task2", 1, 8, 1); // 高优先级
    createTestTask(testTask3, "Task3", 2, 6, 2); // 中优先级
    createTestTask(testTask4, "Task4", 3, 12, 1); // 高优先级
}

/* 测试时间片轮转调度算法 */
void testRoundRobin(void) {
    myPrintk(0x2, "\n=== Testing Round Robin Scheduling (quantum=3) ===\n");
    setSchedulingAlgorithm(SCHED_RR, 3);

    createTestTask(testTask1, "Task1", 0, 10, 1);
    createTestTask(testTask2, "Task2", 1, 8, 2);
}

```



```

| | └─ scheduler.h
| | └─ schedulers.h
| | └─ string.h
| | └─ task.h
| | └─ taskArrv.h
| | └─ taskPara.h
| | └─ taskQueue.h
| | └─ task_old.h
| | └─ uart.h
| | └─ vga.h
| | └─ vsprintf.h
| | └─ wallClock.h
| └─ kernel
| | └─ Makefile
| | └─ mem
| | | └─ Makefile
| | | └─ dPartition.c
| | | └─ eFPartition.c
| | | └─ malloc.c
| | | └─ pMemInit.c
| | └─ scheduleTest.c
| | └─ scheduler.c
| | └─ schedulers.c
| | └─ task.c
| | └─ taskArrv.c
| | └─ taskPara.c
| | └─ taskQueue.c
| | └─ tick.c
| | └─ wallClock.c
| └─ lib
| | └─ Makefile
| | └─ string.c
| └─ myOS.ld
| └─ osStart.c
| └─ printk
| | └─ Makefile
| | └─ myPrintk.c
| | └─ types.h
| | └─ vsprintf.c
| └─ start32.S
| └─ userInterface.h
└─ output
| └─ multibootheader
| | └─ multibootHeader.o
| └─ myOS
| | └─ dev
| | | └─ i8253.o
| | | └─ i8259A.o
| | | └─ uart.o
| | | └─ vga.o
| | └─ i386
| | | └─ CTX_SW.o
| | | └─ io.o
| | | └─ irq.o
| | | └─ irqs.o
| | └─ kernel

```



```

echo "快速调度算法测试 - $ALGORITHM"
echo "===== "

# 检查目录
if [ ! -f "Makefile" ]; then
    echo "错误: 请在src目录下运行此脚本"
    exit 1
fi

# 备份原始文件
if [ ! -f "userApp/main.c.backup" ]; then
    cp userApp/main.c userApp/main.c.backup
    echo "备份原始main.c"
fi

# 设置测试参数
case $ALGORITHM in
    fcfs)
        DEFINE_FLAG="TEST_FCFS"
        ALG_NAME="FCFS (先来先服务)"
        ;;
    sjf)
        DEFINE_FLAG="TEST_SJF"
        ALG_NAME="SJF (最短作业优先)"
        ;;
    rr)
        DEFINE_FLAG="TEST_RR"
        ALG_NAME="RR (时间片轮转)"
        ;;
    *)
        echo "用法: $0 [fcfs|sjf|rr]"
        echo "默认使用fcfs"
        DEFINE_FLAG="TEST_FCFS"
        ALG_NAME="FCFS (先来先服务)"
        ;;
esac

echo "测试算法: $ALG_NAME"

# 准备测试文件
cp userApp/scheduleTestMain.c userApp/main.c
sed -i "1i#define $DEFINE_FLAG" userApp/main.c

# 编译
echo "正在编译..."
make clean > /dev/null 2>&1
make > compile.log 2>&1

if [ $? -ne 0 ]; then
    echo "编译失败! "
    cat compile.log
    # 恢复文件
    cp userApp/main.c.backup userApp/main.c
    exit 1
fi

```

```

echo "编译成功！"
echo "启动测试（按Ctrl+C停止）..."
echo ""

# 运行测试
qemu-system-i386 -kernel output/myOS.elf -serial stdio -display none

# 恢复原始文件
echo ""
echo "恢复原始文件..."
cp userApp/main.c.backup userApp/main.c
rm userApp/main.c.backup

echo "测试完成！"

```

第四部分 运行结果

1.FCFS

```

=====
快速调度算法测试 - fcfs
=====

备份原始main.c
测试算法：FCFS（先来先服务）
正在编译...
编译成功！
启动测试（按Ctrl+C停止）...

MemStart: 100000
MemSize: 7f00000
_end: 10c390
dPartition(start=0x10c390, size=0x7ef3c70, firstFreeStart=0x10c400)
EMB(start=0x10c398, size=0x68, nextStart=0x0)
EMB(start=0x10c400, size=0x7ef3c00, nextStart=0x0)
dPartition(start=0x10c390, size=0x7ef3c70, firstFreeStart=0x10c398)
EMB(start=0x10c398, size=0x7ef3c68, nextStart=0x0)
START MULTITASKING.....
*****
*      SCHEDULING ALGORITHM      *
*          DEMO PROGRAM          *
*****

=== FCFS Scheduling Test ===
Scheduling algorithm changed to: FCFS
Creating task: Task1 (arrival=0, burst=10, priority=1)
Creating task: Task2 (arrival=0, burst=8, priority=2)
Creating task: Task3 (arrival=0, burst=6, priority=3)
FCFS tasks created, execution order should be: Task1 -> Task2 -> Task3
Task1[0] Task1[1] Task1[2] Task1[3] Task1[4] Task1[5] Task1[6] Task1[7] Task1[8]
Task1[9] Task1 END
Task2[0] Task2[1] Task2[2] Task2[3] Task2[4] Task2[5] Task2[6] Task2[7] Task2 END
Task3[0] Task3[1] Task3[2] Task3[3] Task3[4] Task3[5] Task3 END

```

2.SJF

```
=====
快速调度算法测试 - sjf
=====

备份原始main.c
测试算法：SJF（最短作业优先）
正在编译...
编译成功！
启动测试（按Ctrl+C停止）...

MemStart: 100000
MemSize: 7f00000
_end: 10c390
dPartition(start=0x10c390, size=0x7ef3c70, firstFreeStart=0x10c400)
EMB(start=0x10c398, size=0x68, nextStart=0x0)
EMB(start=0x10c400, size=0x7ef3c00, nextStart=0x0)
dPartition(start=0x10c390, size=0x7ef3c70, firstFreeStart=0x10c398)
EMB(start=0x10c398, size=0x7ef3c68, nextStart=0x0)
START MULTITASKING.....
*****
*      SCHEDULING ALGORITHM      *
*          DEMO PROGRAM          *
*****

=== SJF Scheduling Test ===
Scheduling algorithm changed to: SJF
Creating task: Task1 (arrival=0, burst=10, priority=1)
Creating task: Task2 (arrival=0, burst=4, priority=2)
Creating task: Task3 (arrival=0, burst=6, priority=3)
Creating task: Task4 (arrival=0, burst=2, priority=1)
SJF tasks created, execution order should be: Task4(2) -> Task2(4) -> Task3(6) ->
Task1(10)
Task4[0] Task4[1] Task4 END
Task2[0] Task2[1] Task2[2] Task2[3] Task2[4] Task2[5] Task2[6] Task2[7] Task2 END
Task3[0] Task3[1] Task3[2] Task3[3] Task3[4] Task3[5] Task3 END
Task1[0] Task1[1] Task1[2] Task1[3] Task1[4] Task1[5] Task1[6] Task1[7] Task1[8]
Task1[9] Task1 END
```

3.RR

```
=====
快速调度算法测试 - rr
=====

备份原始main.c
测试算法：RR（时间片轮转）
正在编译...
编译成功！
启动测试（按Ctrl+C停止）...

MemStart: 100000
MemSize: 7f00000
_end: 10c390
dPartition(start=0x10c390, size=0x7ef3c70, firstFreeStart=0x10c400)
EMB(start=0x10c398, size=0x68, nextStart=0x0)
```

```

EMB(start=0x10c400, size=0x7ef3c00, nextStart=0x0)
dPartition(start=0x10c390, size=0x7ef3c70, firstFreeStart=0x10c398)
EMB(start=0x10c398, size=0x7ef3c68, nextStart=0x0)
START MULTITASKING.....
*****
*      SCHEDULING ALGORITHM      *
*      DEMO PROGRAM              *
*****

=== Round Robin Scheduling Test (quantum=3) ===
Scheduling algorithm changed to: Round Robin (quantum=3)
Creating task: Task1 (arrival=0, burst=10, priority=1)
Creating task: Task2 (arrival=0, burst=8, priority=2)
Creating task: Task3 (arrival=0, burst=6, priority=3)
RR tasks created, should see interleaved execution with time quantum=3
Task1[0] Task1[1] Task1[2] Task1[3] Task1[4] Task1[5] Task1[6] [RR] Time quantum
expired, switching task
Task2[0] Task2[1] Task2[2] Task2[3] Task2[4] Task2[5] Task2[6] Task2[7] Task2 END
Task3[0] Task3[1] Task3[2] Task3[3] Task3[4] Task3[5] Task3 [RR] Time quantum
expired, switching task
Task1[7] Task1[8] Task1[9] Task1 END
END

```

可以发现，均成功完成了算法调度。